Capstone Project
Expression of Interest

Student Information		
Name:	Ryan Barkman	Mark Seferiades
Student No:	991690389	991697172
Email Address:	barkmanr@sheridancollege.ca	seferiam@sheridancollege.ca
Specialization:	Game Engineering	
Capstone Inception Start Term:	Fall 2025	

Project Overview	
Project Title:	**Last Stand: Oregon's End**
Project Domain:	Entertainment Survival Zombie Video Games Game Modding
Problem-Solving Objective (PSO) (what)	**Many zombie games lack emphasis on survival aspects and proper looting.** While there are big-name zombie arcade shooters like *Call of Duty*, *Left 4 Dead*, and *Dead Island*, there is a gap in the market for survival-focused zombie games as well as single player games. This is a problem where big companies are making similar multiplayer games leaving single player games in the dust. What we're really looking for are games that emphasize resource gathering, decision-making, and strategy, where players must carefully plan and adapt to survive. Games like *Project Zomboid* and the *Last Stand* franchise come closest to this vision, but we need more games that prioritize survival elements over pure action. Our solution is to create a video game to promote this style of gaming and to potentially put our own game on the map. This intern might make a new wave in the single player industry. **Game longevity for single player games:** Another significant challenge for developers is game longevity. After players finish a game, they tend to move on to different releases. First solution is multiplayer as playing real people can make endless content, but the gameplay can be dull if no new content doesn't get regularly added. We are also against multiplier games because most have no souls and are money grabs. A popular solution is achievements, but these don't keep the majority of the player base as most leave when finished. A solution that many games don't fully capitalize on is modding. This Allows players to create and share custom content that can keep a game alive well beyond its initial release. Mods have kept games like *Black Ops 3* and *Left 4 Dead 2* thriving for years,

	offering new experiences and content for players long after the original developers have moved on. *Garry's Mod* is a prime example of this: it has survived for 20 years on community-driven content alone, proving how effective modding can be at sustaining a game's life cycle. Our solution is not to redefine current solutions but to promote existing ones to become a new norm.
Motivation (why)	Ryan has always loved the *Last Stand* series; it really laid the groundwork for zombie games during the early Flash game era. *Last Stand Union City* is in his top 10 games of all time. This game franchise is the main reason why he wanted to make his own version where players are trying to survive while travelling across the country. Ryan believes that the *Oregon Trail* is the perfect medium to add to this idea as its travel survival is very applicable to the *Last Stand*, where players would have to survive while making their way across the country with random encounters. These types of games are apart of genres that don't get nearly as much love as it deserves. Mark also appreciates zombie survival games such as the *Left 4 Dead* Series and *Call of Duty* zombies. Implementing more hardcore survival mechanics seems to be very rewarding for players. Mark also believes that implementing strong modding support is the best path forward for improving the game's longevity. Some of his favourite game series, such as Company of Heroes and Total War have extensive mod support, their older games have benefitted greatly in the long run because of it. Modding also can inspire people to become game programmers and developers, thus is good for the entire game industry. When it comes to game longevity, modding was a much-loved aspect in favorite games. Games like *Black Ops 3* and *Left 4 Dead 2* remain entertaining, not just because of their base content, but because of the mods that add new experiences that extend their lifecycles. Mods keep the games fresh, allowing players to explore new content, try different gameplay styles, and enjoy an endless variety of experiences. We have had fond memories of playing Flash games with cheats to give more freedom to the players in creative ways, and we think that more games should incorporate mods to allow easy access to enhance experiences.
Solution Description (how)	**A Blend of The Last Stand and Oregon Trail Remake** Our plan is by combining the survival decision-making and travel from *The Oregon Trail* remake, with the looting and combat mechanics from *The Last Stand* franchise (*1,2 and Union City*). In *Oregon Trail*, players embark on a long journey across the United States, making decisions that affect the survival of their party. Along the way, they face random encounters that could alter the course of their journey. In *The Last Stand*, players loot abandoned houses, collect weapons, and fend off hordes of zombies, crafting a fun mix of survival and action. By merging these two concepts into a 2D survival game, where players must travel on foot from the Oregon to the West Coast of the United States. The journey will be filled with random encounters, hostile enemies,

	resource management, and more that force players to make tough decisions to ensure their survival. Just like in *The Last Stand*, players will be able to loot towns to gain useful tools to help get them to the next town. Players must decide when to take risks and when to conserve resources with enemies constantly threaten progress, forcing players to use their limited. This simple yet fun gameplay would create a unique survival experience, where every choice could mean life or death. Mods will be able to be imported in the game with unity package that can modify the experience of the game. These can include different skins for the weapons and player, but also includes adding new content like new weapons, encounters, and enemies. This will either be done within the game or through pre-made features like steam workshop. An advanced mod manager will be implemented that streamlines the use of multiple mods, handling dependencies and conflicts.
Stakeholders (who)	**Gamers (primary actor):** Primary audience for the game. These are the people who play the game which we want to keep on and keep entertained to generate positive feedback and potential recommendations for growth. Targeted genres include Zombies, Survival, Flash, and Mods. **Modders (primary actor):** People who enjoy the game enough with development experience to create mods for other players to enjoy as well. These people also prolong the game's life.
Domain Expert / Domain Research	Games that support modding often enjoy a much longer lifespan compared to those that don't, largely because they foster a thriving community of creators and players who continually add to the game. One of the best examples of this is *Garry's Mod*, a game that has maintained a massive and active player base for over 20 years. Its ability to adapt and evolve through community-created content has kept it relevant and engaging, allowing it to survive well beyond the typical life cycle of most games. Similarly, *Call of Duty: Black Ops 3* is another prime example of the power of modding. Despite being released years ago, it still boasts a massive player base on Steam, outpacing even newer titles in the franchise. The game's modding community plays a key role in this, as custom content and new mods have kept the game fresh, appealing to both veteran players and new ones alike. By allowing mods, games can benefit from continuous content updates, keeping them fresh and relevant long after the official development phase has ended, leading to a sustained and engaged player base. Zombie survival games have been showed to be successful with their reviews, *Project Zomboid*, *Last Stand*, *Last of Us* and *Fallout* are all successful looting games. While great reviews, some are not mainstream due to single player gets out shined by multiplayer. There still needs to be a space for single player games as they can have amazing stories, gameplay, and unique experiences where you don't need connections to play.

Equipment and Software Needs	Hardware: ○ Powerful enough computer to develop games in Unity Engine ○ Already acquired, as per our program's requirements Software: ○ Unity game engine ○ Visual Studio 2022 ○ Cloud Infrastructure and API, either of these 3: ○ Unity Cloud ○ Steamworks SDK ○ Custom cloud using AWS, Azure, or Google, with RESTful API ○ Version Control ○ Git and GitHub ○ Unity Version Control

Technical Requirements

Game Development	In this 2D survival game, players begin their journey in Oregon and must travel across the United States, heading east to reach the east coast to win the game. As players move from state to state, each travel segment takes time, allowing for random encounters to occur along the way. These encounters could range from positive discoveries, like finding useful supplies, to dangerous situations, such as being ambushed by zombies or having to reroute due to an impassable obstacle. Every choice and route taken could influence survival. Entering cities provide the best loot opportunities, with a variety of resources: such as weapons, medical kits, and ammunition. Cities also serve as points of interest where players may encounter other survivors, both hostile and friendly. Cities are shown on the map, but players may be able to find secret cities through encounters. During combat players will control the character: moving, aiming, and shooting at enemies. The combat will focus on resource management, requiring players to balance offense and resource management. Using Unity, modders will have the ability to create custom packages that can be imported into the game. This will allow players to create and share their own weapons, skins, enemies, and even new encounters. Formats will need to be provided to aid with player creations.
Cloud Technology	The system will leverage advanced cloud technology to manage and store saved games, encompassing player progress, and full game states by integrating both local and cloud saving capabilities. Off-the-shelf solutions like Unity Cloud Save and Steam Cloud offer seamless integration via their SDKs and robust data management with automatic backup and security features, though typically tied to player accounts. Alternatively, a custom solution built with AWS, Azure, or Google Cloud would allow an account-less approach (using a username, password, and game name) by designing RESTful API endpoints and a scalable backend (with load balancing, caching, and managed

	databases) while employing advanced encryption and rigorous access control. This strategy not only ensures reliability, scalability, and security under heavy data loads but also satisfies the advanced areas of computer science requirement by implementing cutting-edge distributed systems and secure data management protocols within the game.
CS Advanced Areas	**Dynamic Mod Manager (Dynamic Dependency Resolution):** This feature automates mod loading by treating each mod as a node in a dependency graph, with interdependencies resolved through topological sorting and constraint-solving algorithms. It handles complex mod interactions and version conflicts while enabling real-time integration of new features using advanced techniques like dynamic linking and runtime reflection. This system showcases sophisticated modularity and extensibility, creating a flexible ecosystem that seamlessly incorporates community-driven content into the game. **Advanced Ballistics Simulation (Computational Physics):** This feature involves the use of computational physics and numerical methods to model projectile trajectories, accounting for gravity, drag, and material interactions. It employs numerical integration techniques to update a bullet's position and velocity in small increments, simulating energy loss, penetration, and deflection more accurately than basic ray-casting. This approach, which involves solving differential equations and implementing custom collision response models, delivers an immersive gameplay experience that balances realism with computational efficiency. **Entity AI:** Enemies and companions will utilize advanced AI features based on techniques learned in relevant courses. Navigation meshes (NavMeshes) will be adapted to support 2D gameplay, allowing characters to intelligently jump to higher platforms in the absence of stairs. AI behavior will also dynamically respond to the player's position, adjusting movement if the player gets too close or too far. Combat logic will include predictive targeting, aiming at where the player is likely to be rather than where they currently are.

Other Information

Prior Experience	**Ryan:** I currently do not have direct experience with projects of this scale, especially in areas like cloud capabilities and modding. While I have worked with small Unity projects in the past, I have not yet explored integrating cloud services or handling large-scale data, though I plan to touch upon cloud technologies in my upcoming Software Engineering course. However, the course will cover much smaller data sets compared to what will be needed for this project. In terms of modding, my experience is limited to downloading mods from platforms like the Steam Workshop, but I have not yet worked on creating or integrating custom mods myself.

	While I don't have extensive experience in these specific areas, I have a strong vision for the game and am eager to learn and expand my skill set in these new areas. I am committed to gaining the necessary expertise to successfully execute this project. **Mark:** My prior experience is similar to Ryan's. I have no real experience with large scale projects (games) like this. I have real experience when it comes to cloud integration in games. Another area that I have limited experience with is integrating modding support into games. I do have experience working on various smaller projects (games) in Unity. In addition to this, I have experience creating game mods, with various modding tools, and uploading them to the steam workshop (where mods are hosted and managed in Steam's cloud). Currently, I'm taking two game engineering specialization courses that are heavy in Unity, further bolstering my experience in Unity. Furthermore, I'm working on a group project (a game) using Unity in the Software Engineering course, that also implements cloud features for account authentication and management. This experience will not just be important for the technical challenges of cloud integration, but also team experience using Unity.
Conflict of Interest	**Potential Legal Issues (Copyright Concerns):** • There is a risk of legal action if the game is too similar to the work it inspires from like *The Oregon Trail* or *The Last Stand* franchises. If the gameplay or mechanics are too close to these source materials, the game could be threatened with copyright infringement. • **Solution:** To mitigate this risk, the game will incorporate unique features, such as a distinct set of encounters, different enemy types, and unique art style, to differentiate it from the original games. This will ensure that the game feels original while still drawing inspiration from those titles. **Anti-Support from Large Gaming Corporations:** • Some big gaming companies, like those behind *Call of Duty*, discourage modding, as they prefer players to focus on newer releases rather than revisiting older titles. Companies like *Nintendo* may also resist modding, fearing that fan-made content could damage the integrity of their intellectual property. This doesn't affect the project, just want to bring awareness to games being pro mods.

Introduction

This game is designed to provide a fun and immersive experience for players who enjoy single-player survival games. It is particularly targeted toward those who have a connection to classic flash games, especially those who grew up playing on platforms like Armor Games and Newgrounds. These players, who are probably young adults, are willing to play unique titles compared to fast-paced multiplayer games which are the current big titles. The game blends elements from The Oregon Trail and The Last Stand franchises, combining survival mechanics with fun combat. Players will journey across the U.S. in a

post-apocalyptic world, scavenging for supplies, fending off enemies, and making decisions that impact their survival. The game emphasizes strategy and resource management, with challenging random encounters along the way forcing player to use their resources. Game data will be stored on the cloud by the game, allowing players to continue their playthroughs across multiple devices. Mod Support will also be an implementation, allowing players to add custom items using Unity Packages for game longevity.

Problem-Solving Domain (Where)

Zombie & Survival Genre:
While titles like *Project Zomboid*, *Oregon Trail*, and *Last Stand* have earned strong reviews, they often remain under the mainstream radar in the broader gaming industry. Most major zombie games tend to focus on arcade-style action or first-person shooters. While these games can achieve critical success, there's still a gap for a more immersive, resource-focused survival experience; one that goes beyond just run and gun mechanics. Our goal is to fill the gap by reviving the nostalgic feel of Flash-era survival games while reintroducing the core elements of zombie survival: managing resources, making strategic decisions, and survival; want a view of an unforgiving world. We want to bring back the genre, one where planning, survival, and exploration are just as important, if not more so, than combat. The closest mainstream release that comes close to this vision is The Last of Us on grounded difficulty where players are clutching to any resources they can and the Fallout series where you can loot lots of locations. There is something so fun about looting as much as you can.

Game Longevity:
One of the most powerful ways to extend the longevity of a game is by providing support modding. Games that allow player-created content, tend to have a significantly longer lifespan, keeping their player base active many years after the initial release, even when other games in the same genre may have "died".

- **Left 4 Dead 2** (2009):
  Averaged 45,000 players per month last year, proving that old games can maintain a strong player base with modding support.
  https://steamdb.info/app/550/charts/

- **Call of Duty: Black Ops 3** (2015):
  Averaged 10,000 players per month last year. While not as high as Left 4 Dead 2, it still shows the impact of modding on maintaining a player base.
  https://steamdb.info/app/311210/charts/#1y

- **Call of Duty: Modern Warfare 2019**:
  Averaged just 1,500 players per month last year. Despite being a newer release, it doesn't have modding support, highlighting the significant difference mods can make in sustaining a player base.
  https://steamdb.info/app/2000950/charts/#1y

- **Garry's Mod** (2006): Averaged 35,000 players per month last year. Its modding community is a central factor in its lasting popularity
  https://steamdb.info/app/4000/charts/#1y

These examples highlight the role modding plays in prolonging a game's lifecycle. Allowing players to create custom content, from new maps to weapons, helps keep the game fresh, engaging, and relevant, even years after its original release. Games like *Garry's Mod* exemplify this, where the modding community's creativity continually breathes new life into the game. In our opinion, *Garry's Mod* will likely never see a significant drop in its player base unless a direct sequel is released. The active modding community and the game's inherent versatility are key reasons for its enduring popularity. By supporting mods, more games can replicate this success, not only keeping the existing player base engaged but also attracting new players who enjoy the creativity and customization mods bring.

Techniques such as procedural level generation have been explored to improve a game's longevity by improving replay-ability. The idea behind this is for the game to use an algorithm to automatically generate a random level for the player that makes sense and aligns with the other levels. An example of a game that does this is the Diablo series, having procedurally generated dungeons. But players have noted issues with this method. One of the problems is that the algorithms aren't perfect, and clear patterns emerge from the level generation. The layouts therefore become easier to predict, and less enjoyable. Another problem is emotional disconnection. The lack of thoughtfully placed objects and landmarks by humans tends to make the atmosphere feel generic. Procedural level generation as of now can't be used to form a narrative or atmospheric story telling. It reduces the emotional impact, and therefore the player engagement.

Modders will be able to craft handmade levels and experiences that have a human touch that algorithms cannot yet replicate as noted above. In addition to this, modding will allow for not only new experience in the form of levels, but also other content, such as weapons, armor, equipment, character skins, new characters. There is far greater possibility for improvement and expansion with modding compared to procedural generation.

Problem-Solving Objective (What)

1. Game Longevity (Challenges with Keeping Games Alive):
One of the main challenges for game developers is keeping a game alive once its player base begins to dwindle after players have completed the main objectives. As the game becomes less active, it can quickly turn into a "dead" game, with players moving on to newer titles. While some companies attempt to extend the lifespan of their games by adding additional content, features, or creating longer experiences, these efforts often require significant resources and may not always yield a good return on investment. Common strategies like adding achievements to encourage players to complete the game 100% are often used. However, this approach rarely retains most of the player base as most players don't care about achievements. Once players complete the core objectives, many leave, and only a smaller, more dedicated player base remains to fully complete it.

The Best Solution: Modding:
A more effective, yet often underutilized, solution to this problem is modding. By allowing players to create and share their own custom content, a game's lifecycle can be extended indefinitely. Mods bring fresh, community-created experiences to the game, keeping it dynamic and engaging for players long after the main content has been completed. Players can download new mods, explore custom content, and share their own creations, ensuring a continuous influx of fresh material and creativity. This solution comes with its own challenge: mod security. Malicious or harmful mods could compromise the game's integrity and the overall player experience. To address this, we might need to implement robust moderation and security measures to safeguard against any potential risks.

The Fun and Unexpected:
Mods also introduce unique experiences that were not initially intended by the game's developers. For example, *Left 4 Dead 2* features mods that turn the game's once-scary zombie apocalypse into absurd and humorous scenarios by transforming zombies into *Fall Guys* characters. This provides a lighthearted twist thar can even bring in different players as it adds a new layer of fun to the game, showing how mods can radically change the tone and gameplay experience.

2. Lack of Survival Games (A Personal Preference with Potential)
While this is more of a personal preference, it's clear that there is a noticeable gap in the market for zombie survival games. Games that emphasize resource management, survival, and decision-making have broad appeal but are underrepresented in some circles. There's significant potential for growth in this niche, especially when considering the growing interest in survival mechanics across gaming communities. The success of indie games has proven that even without a massive initial fanbase, a game can achieve significant commercial success. Titles like *Project Zomboid, Among Us, Stardew Valley,* and *Lethal Company* started as indie projects but achieved remarkable success. These games resonated with players not just because of their gameplay mechanics but because they tapped into unique, niche experiences. Expanding on the survival genre, especially in the context of a zombie apocalypse, could lead to success, tapping into a market hungry for new, engaging survival experiences. The key is to find a fresh take on survival gameplay that appeals to players who enjoy both strategy and action but are looking for something new and unique.

Motivation (Why)

Ryan:

I've always been drawn to games that focus on survival and strategy, but unfortunately, these genres often don't receive the attention or appreciation they deserve. The *Last Stand* series was a staple of early zombie game success, especially with their Flash games, but their more recent works haven't gained the same level of recognition. While still excellent games, they aren't as widely talked about, especially in comparison to big multiplayer franchises. This, to me, is a problem worth addressing because every genre should have its place, and players should have the freedom to choose the types of games they want to play. Diversity in gaming experiences is essential. When it comes to modding, I truly believe it's one of the most impactful features in the gaming world. Mods can bring fresh life to a game, extending its longevity, and offer players a chance to add their own creative touch to something they love. However, modding faces opposition from two major types of corporations that often hinder its full potential. The first group are the mass producers. Companies like *Call of Duty* thrive on yearly releases and rely on players constantly moving from one title to the next: maximizing profits. They don't care about prolonging the life of their games; they want the next big thing to sell. For these companies, modding is a threat because it removes the pressure of constantly needing new content or sequels. The second group are the "perfectionists". Companies like Nintendo, who tightly control their creations. They believe that any changes or additions made by the community undermines their vision and tarnishes the experience they've meticulously crafted. This mindset leads to a disregard for the community and a reluctance to embrace mods. That's why I'm passionate about modding. Embracing mods means a company values their work, wants it to last, and truly cares about their fanbase. It shows a level of respect for the community, allowing them the creative freedom to add something new, unique, and exciting; something the developers may never have imagined themselves. I believe mods should be a staple in video games and wish to spread the message. One big company that I believe is

pushing in the right direction is Epic Games with Fortnite as it allows players a lot of freedom for custom creations as I have used it to create functional board games with their creative mode.

Mark:

I have enjoyed zombie survival games such as the *Left 4 Dead* Series and *Call of Duty* zombies, as many other players have. Players tend to be drawn to the zombie survival genre, but there tends to be a lack of survival elements, at least from games by larger studios. Implementing more hardcore survival mechanics seems to be very rewarding for players. They can offer increased challenge and tension, as well as a greater sense of player agency and accomplishment.

I also believe that implementing strong modding support is the best path forward for improving the game's longevity. Some of his favourite game series, such as Company of Heroes and Total War have extensive mod support. Some of their games older games have benefitted greatly in the long run because of it. This includes games that are over 10 years old now, such as Rome 2 Total War and Company of Heroes 2. These games despite their age have impressive player counts today, all because of their expansive modding communities. Modding also can inspire people to become game programmers and developers, thus is good for the entire game industry.

Solution Overview (How)

Game Features:

- **Traveling:** There will be two travelling modes: state travel and city travel. State travel allows players to pick different cities or empty areas on the U.S. map to travel to from their current position, trying to get closer to the end goal. Getting close to a city allows them to enter and start city travel mode where they can search for loot by selecting unknown buildings to travel too; when finished they can go back to state travel to get closer to end destination. Building starts off as unknows, but players may be able to discover building types through encounters. Players won't be able to travel through big water areas, forcing them to go around. For increased efficiency, players can use established paths between cities that will be faster than going rough and travelling towards the end goal alone. Each second in between picked destinations are in walking mode where each second you are moving closer to target but also can trigger a random event that can affect progress.

(Last Stand 2 photo demonstrating what state travel can look like, will be much bigger as players are going across America and not just a state).
https://static.wikia.nocookie.net/tlaststand/images/b/bb/TLS_2_MAP.jpg/revision/latest/scale-to-width-down/290?cb=20120325092622

(Last Stand 2 photo demonstrates what city travel can look like. Emtyp at first but may find symbols)
https://static.wikia.nocookie.net/tlaststand/images/7/7d/Jonestown.jpg/revision/latest?cb=20101030113530

- **Encounters:** Random events that can impact the player's journey while traveling. These events are drawn from a dynamic pool, which is determined by factors like the current zone, state, and city. Pool has an initial l list but each of the criteria will add more to the pool. Each area contributes to a large list of possible encounters, which will be randomly selected when an event triggers during the travel process. When an encounter occurs, it temporarily halts the traveling process until the player resolves the event, after which they can continue their journey. The types of encounters include but are not limited to are:
    - **Hostile Encounters:**
      These can include zombie hordes, bandits, or other dangerous adversaries that the player must face to survive. This can switch the player into combat mode allowing control of the character.
    - **Positive Encounters:**
      Players may come across free loot on the side of the road, or stumble upon hidden cities that aren't visible on the map, offering additional resources or opportunities.
    - **City-Based Encounters:**
      When traveling in or near a city, encounters may include discovering building types pointing to hospitals or gun stores, can find survivors willing to join the player's group.
    - **Risk vs. Reward Choices:**
      Players may face a decision to take a risk for a chance of valuable loot. These choices can lead to rewards but also carry the potential for danger.
    - **Information Encounters:**
      Players might find info on the ground that may help but not anything that affects them immediately.

  These encounters will enhance the unpredictability of the game, ensuring that each journey feels unique and keeping players on their toes throughout their survival experience. Also helps with game longevity.

- **Zones:** The game world will be divided into different zones. Each zone consists of a group of neighboring states. These zones influence the types of encounters that players will face during their travels making some areas harder than others. Each will have pros and cons where some may be shorter but result in harder survival. For example: A zone could be dominated by zombies where encounters will more likely be zombies, making it a high-risk area where combat and resource management are crucial. Another zone could be controlled by a zone specific raider faction, leading to encounters with hostile human groups that might ambush the player or offer high-stakes decisions on whether to negotiate, fight, or flee. Each zone adds its own set of encounters to the broader pool of random events. When a player travels through a zone, the game pulls from this customized pool, ensuring that encounters feel unique to the region. This system will not only make each area more dynamic but will also encourage players to adapt their strategies depending on the zone they are in.

(This can be what zones look like where each zone results in different encounters)

- **Weapons:**
  Guns:
    - **Bullet Damage**: The amount of damage a bullet deals upon impact. Higher damage values make weapons more lethal but may reduce ammo efficiency.
    - **Rate of Fire**: The time between each bullet fired. A faster shoot speed allows for more rapid fire, while slower speeds require more precise shooting. Won't affect semi-autos.
    - **Damage Range**: The distance at which the weapon's bullet damage will drop off (e.g., half damage at a certain distance). This determines the effective range of the weapon.
    - **Vision**: The player's view ahead of them while aiming or shooting. Weapons with a wider field of view help with seeing more enemies or objects in the distance.
    - **Reload Speed**: The time it takes to reload the weapon after the magazine is emptied. Faster reload speeds can be crucial in combat scenarios.
    - **Wait Time**: The number of seconds it takes before the next burst of bullets can be fired. This cooldown period affects burst and semi auto weapons.
    - **Burst**: The number of bullets fired in a single burst before the player must wait (related to the Wait Time).
    - **Clip Size**: The maximum number of bullets the weapon can hold in a single magazine before a reload is needed.
    - **Ammo Type**: The type of ammunition used by the weapon. Related to resource management.
    - **Semi-Auto**: A Boolean value determining if the weapon is semi-automatic. If true, the player must click to fire each bullet.
    - **Weight**: Affects inventory holding capabilities.
    - **Fired Bullets**: For shotguns that fire multiple pellets at once.
    - **Penetration:** bullets can penetrate zombies or certain objects, with a coefficient to how much energy is lost as the projectile travels through penetrable materials
    - **Attachment Slots**: List of available attachments that can modify stats.

  Melee Weapons:
    - **Damage (Dmg)**: The amount of damage the melee weapon inflicts with each strike.
    - **Attack Speed**: How quickly the player can swing the melee weapon.
    - **Weight**: Affects inventory holding capabilities.

- **Attachments:** Tools that can be added to weapons that directly affect stats. (e.g., scopes, silencers, extended mags). Some will only be applicable to certain weapons.

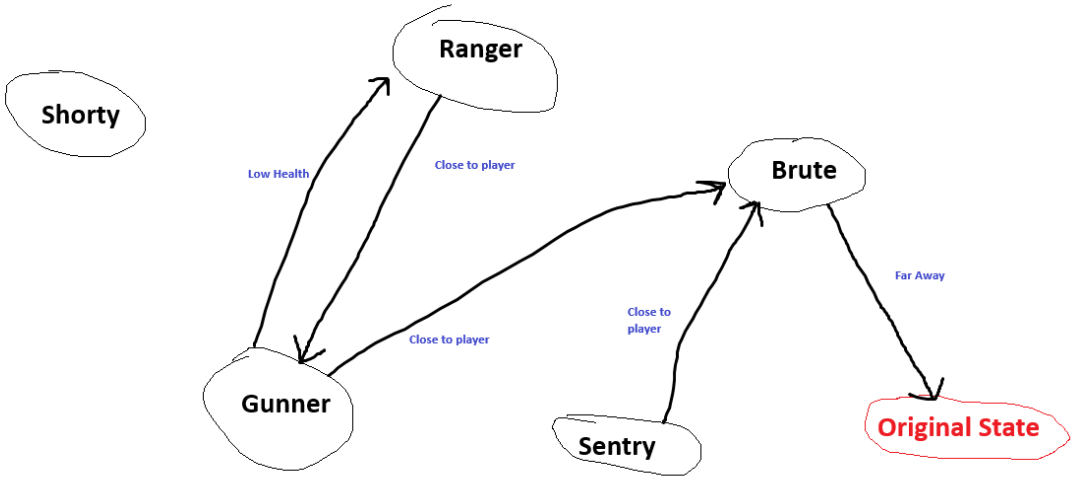
- **Combat:** Player can move around with keyboard and shoot enemies with mouse Enemies can hurt players and party where players need to use their resources to help survive.

- **Zombies:** Core part of enemy encounters and combat. They are designed with three key stats: Health, Speed, and Damage. Their AI is intentionally simple: braindead and aggressive; focused solely on charging directly at the player without any strategic thinking. They use Navigation Meshes to assist with movement and ensure they can reach the player effectively, even around obstacles. Zombie Types Include:

	Health	Speed	Damage
Zombie	Normal	Normal	Normal
Runner	Low	High	Normal
Heavy	High	Low	High
Tank	Very High	Very Slow	High
Bleeder	Normal	Low	Very High
Weaker	Low	Normal	Low

- **Enemies:** Enemies play a key role in combat encounters, actively attempting to harm the player. Their behavior is governed by finite state machines, allowing for dynamic decision-making based on their type, distance to the player, and current health. Each enemy starts with their own state but allows change. Will keep track of original state to go back to. Enemy Types:
    - **Shorty** – Prefers close-range combat, getting near the player to fire. Typically equipped with shotguns.
    - **Ranger** – Fights from a long distance, keeping away from the player while shooting.
    - **Gunner** – The default enemy type. Maintains a moderate distance while engaging the player.
    - **Sentry** – Stationary enemies that remain in one spot, firing from their fixed position.
    - **Brute** – Melee-focused enemy that charges directly into close-quarters combat.

  Each enemy type adjusts its behavior dynamically switching states based on factors like **health** and **proximity to the player**. Navigation Meshes will support their movement, allowing enemies to navigate terrain intelligently. Swapping to brute equips melee and re swaps when going back to original state.



- **Cities:** randomly generated from list of cities when entering a new city, Players will be able to search in unknown buildings to gain loot. Different building types will result in different loot pools. New encounter types like directions to buildings like a hospital or police station can happen. Players are not getting closer to their end destination in a city; just loot gatherings.

- **Looting:** Looting will allow players to search in buildings for vital resources. The loot pools vary depending on the type of building being searched. Certain items may be more likely to be found in specific locations. Types of Loot:
    - **Weapons**: Guns, melee weapons, ammo, and attachments.
    - **Supplies**: Water, food, health items, scrap (general stuff), money.
    - **Player Upgrades**: Items that improve player stats like health or strength.

- **Player Stats:** The player has several key stats that will impact gameplay. These stats can be improved through leveling up or looting. **Stats:**
    - **Health**: Determines how much damage the player can take before dying.
    - **Speed**: Impacts traveling speed, and speed in combat mode.
    - **Hunger & Thirst**: Requires the player to manage food and water intake to survive.
    - **Strength**: Affects the player's ability to carry items and deal damage in melee combat.
    - **Level**: The overall progression of the character. As the player levels up, they can enhance their stats.
    - **Dexterity**: reload speed, melee speed
    - **Charisma Luck**: Chance of peaceful encounters

- **Inventory:** The inventory is limited by weight. Each item in the game has a weight value, and players can only carry as much as their strength statistics allow. Players also have two weapon slots that don't affect the weight but hold weapons that can be easily accessed in combat. Items will have a category associated with them to allow finding.

- **Survivor:** Survivors are NPCs that will accompany the player, helping in combat and providing additional support. Survivors have their own health and can die during encounters. If the main character dies, control can shift to a survivor by choice, ensuring continuity in gameplay. Stats are set back to default if switched. Each party member will take a bit of food when player consumes leading to less to them. Won't take health unless given directly to them. They have a lot more health than the player to make sure they don't die easily and only have a gun inventory slot with unlimited ammo. Max = 2.
    - AI Behavior: Stay close to player, if an enemy is within range shoot at closest. Includes Navigation Meshes to stay close to the player when a bit far and if outside certain range teleport to player to ensure not getting lost.

- ~~Difficulty (Mark for Deletion): The game will feature 3 difficulty modes that impact both the environment and enemy behavior. As the difficulty increases, the game introduces additional challenges like managing hunger, thirst, and weight. Difficulty Modes:~~
    - ~~Easy: No hunger, thirst, or weight limits. Enemies deal minimal damage.~~
    - ~~Medium(default): Introduces hunger and weight management. Enemies deal moderate damage.~~
    - ~~Hard: Adds thirst as a resource to manage. Enemies deal increased damage, making survival even more difficult.~~

Modding:

- Players can submit custom creations with unity packages which can be imported in-game. These can be any of the items listed above and will allow new experiences. An example of this can be that a player adds a minigun to the game with a new ammo type that can be found.
- To allow ease of use modding, all our features stated above need to be created through abstraction to allow simple following of the code. In addition to this, objects need to be in groups of pools that allow simple addition to keeping modded features close to premade features (i.e. if a player mods a new weapon they can follow a base class which will then add it to a pool of weapons).
- Areas that we will provide easy guides/code packages include: Encounters, Enemies, Player, weapons, accessories, lootable items, maps, survivors.
- A mod manager will be incorporated, that works in game runtime. It handles the load order and application of multiple mods, dealing with conflicts between mods as well as dependencies that mods may have with modules of the game, or possibly other mods

Project Impact

While the game may not immediately revolutionize the entire gaming industry, it holds the potential to leave an impact on the survival genre, especially within the zombie niche. Similar to titles like *Project Zomboid*, the game doesn't need to be a AAA production to gain recognition. The key is creating something that resonates with players, offering a unique experience that stands out in the crowded genre and becoming a go-to title for survival game enthusiasts. One of the challenges we face is timing. Titles like the *Last Stand* and *Oregon Trail* were not only successful in their own right but were also products of their time, which made them even more iconic. *Last Stand* thrived during the Flash era, becoming one of the top-tier games in that space. However, with the death of Flash, many players who enjoyed those types of games have been left searching for a worthy successor. The emotion that we are trying to get out of gamers is nostalgia; that is our best way to success. So, by reviving elements from that era, with new a modern twist, we have the opportunity to fill this gap. Our goal is to deliver a game that provides the same sense of enjoyment and community, while offering a fresh take on familiar mechanics. If implemented correctly, this game could tap into the nostalgia of former Flash gamers and offer a unique experience for a new generation of players. Not only could players enjoy the game, but they might also recommend it to friends, potentially driving organic growth through word-of-mouth. Through our approach, we aim to cultivate a dedicated player base that could spark a ripple effect within the community, encouraging others to explore the game and contribute to its growth.

The impact we hope to achieve with mod support is to shift the general perception of mods within the gaming industry. We're not aiming to reinvent the wheel, but rather to promote its use. By incorporating mods into our game and offering full support, we could help change how gaming studios and the general gaming population to view modding and its potential in a more positive light.
Our goal is to inspire as many titles as possible to embrace easy modding access, so players can tailor their gaming experience to their preferences. Even if only one game decides to include mods, it still can be a significant win. This single success could encourage other developers to follow suit; spreading modding support across multiple titles. Additionally, we want to foster a more positive relationship between gamers and developers, as mods can empower communities and enhance player respect for the companies behind the games. We believe that when companies embrace modding, they can shift the narrative from being seen as profit-driven to being viewed as more community-focused. For example, companies like Nintendo, despite their incredible games, sometimes face criticism due to their actions, which can make them appear less in tune with their fanbase. Embracing modding could help

companies build stronger, more respectful relationships with their communities and bring more players through positive viewings.

Project Feasibility

The project features a simple design due to its 2D format, which allows for straightforward coding and development. The main challenges will involve handling cloud game data, integrating mod support, and managing the overall time required to meet all the project's requirements. However, the simplicity of the game's mechanics, akin to flash games, makes the construction more manageable overall.

Deployment will be straightforward if the game is developed properly. It can be easily published on platforms like Steam once completed or offered as a downloadable file via a service like Mega. The primary concern here will be ensuring proper security measures are in place, particularly for mod content as it could include harmful data.

Adoption will largely depend on visibility. If published on Steam, the game will have a better chance of being discovered by players searching for new indie games. Additionally, recommendations from players can help spread the game further, creating a viral effect as friends share the game within their communities.

Stakeholders (Who)

Gamers (Primary Actor):
Gamers are the primary audience for this game. These individuals have a deep passion for video games and are always on the lookout for fresh content to engage with. They constantly crave immersive, challenging gameplay that satisfies their desire for entertainment and dopamine boosts.
Targeted Genres:

- **Zombies**: Players who enjoy battling the undead in survival scenarios.
- **Survival**: Gamers who seek out experiences where resources must be carefully managed, and every decision impacts the outcome.
- **Flash Games**: Those who appreciate quick, accessible gameplay, often with minimal system requirements.
- **Mods**: Player who love customizing the experience by adding new content to games, ensuring replay ability and community-driven longevity. These people can also love goofiness by adding the most brain-rotted mods for heavy laughter

Role:
These players will drive the success of the game, as they are the primary audience who will experience and engage with the content, creating demand for updates, new challenges, and community-driven experiences.

Functional Requirements Gamers:

- **Entertainment:** Game needs to be of entertaining value to ensure players don't leave as gamer may leave If it's not entertaining.
- **Gameplay:** Game needs to give curtain level of control to player making sure they are playing the game versus watching. Combat helps this by giving more control to the player by adding input movement and shooting capabilities.
- **Understandable/Learnable mechanics:** mechanics need to be both enjoyable to be used by the player but also learnable to give players a sort of accomplishment / master events

- **Suitable length:** Game needs to have a certain length of gameplay to make sure people have enough time playing. Mods will also boost length.

Modders (Primary Actor):
Modders are players who not only enjoy the game but also possess the development skills necessary to create custom content for the game. They may build new weapons, enemies, maps, or entirely new gameplay features. Their contributions extend the life of the game and add significant variety, enhancing the experience for the entire player base.
Role:
Modders play a crucial role in prolonging the game's lifecycle. Their contributions to the game's content can keep the player base engaged long after the base game has been released. Additionally, modding fosters a community of creativity, allowing players to further immerse themselves in the world of the game.

Functional Requirements for Modders:

- **Ease of Modding:**
  - The game needs to be easy and accessible to modify, through its tools and/or methods.
- **Powerful Modding Capabilities:**
  - The game must have as many mechanics be modifiable as possible.
- **Easy Mod Management:**
  - The game should use a platform that makes it easy for modders to upload, update and manage their mods, as well as be accessible to players who want to use mods

Project Requirements

For networking it includes a cloud database to store game data. It solves problems in the gaming industry by providing mod support to solve the problem of game longevity. Also providing a unique game experience which is fun and different. The project is heavily researched with a clear vision of its end result, which will guide its development. It is feasible to create within the given timeline, as it will not require complex graphics but rather focus on simple, flash-style mechanics that appeal to players who appreciate nostalgic, slower-paced games. Additionally, the project touches on advanced gaming topics such as modding and cloud storage, providing an opportunity to explore these elements in a meaningful and effective way. The combination of these features makes the game both innovative and technically grounded, creating a foundation for success while also solving existing challenges in the gaming landscape.

Game Development Requirements

The game will feature simple 2D graphics with a nostalgic flash-era style to capture the charm of older games. Each location will have unique backgrounds to reflect its environment, enhancing the game's atmosphere and immersion. Combat will be engaging and accessible, allowing players to shoot at enemies using the mouse to select targets or locations on the screen. This approach provides a fun and intuitive shooting mechanic, while maintaining the simplicity of 2D gameplay. Maps will update dynamically to reflect player progress, encouraging exploration and adding to the immersion. Cities will serve as critical points for looting, offering a variety of resources and weapons needed to survive. Additionally, random encounters will be designed so that different regions and locations produce

distinct challenges, rewards, and experiences, making each playthrough unique and unpredictable. Mods allow custom features to enter the mix like guns and unique challenges.

Cloud Technology Requirements

The main use case to fulfill the cloud technology requirement is managing and storing saved games. A system will be developed to ensure both cloud and local game saving, for both player progress and game states. The solution must be reliable, scalable, and secure. In addition to this, we are exploring the possibility of an account-less system, where cloud saved games require username, password, and game name, instead of the player needing an account.

There are several existing solutions designed for games that the project can utilize and expand upon, including Steam Cloud and Unity Cloud Save. These cloud services are easy to integrate and have features that ensure reliability, such as automatic data backup. Implementing either will involve utilizing an SDK (Steamworks SDK for Steam, Unity Cloud Save SDK for Unity) and API (RESTful API for Unity, Steamworks Cloud API for Steam) for transmitting data between the cloud. Both have robust cloud infrastructure ready to go, with many built-in data management and security features. They seem to be more limited in terms of customizability and have the requirement of requiring player accounts.

Alternatively, we can implement a custom solution through cloud services like AWS, AZURE, or Google Cloud Storage, which offers increased flexibility and allows for an account-less saving system, only requiring username, password, and game name. This custom approach would involve designing RESTful API endpoints for data upload and retrieval, implementing a backend data management layer (ideally using a managed, relational database), and establishing a scalable architecture with load balancing and caching to handle large volumes of data efficiently. Such a system may also afford granular control over authentication and file management and allow for seamless integration with the modding framework that the project will implement. In terms of security, adhering to best practices, such as encrypting data in both transit and at rest, and strong access protocols must be utilized.

Advanced Areas of Computer Science

Dynamic Mod Manager (Dynamic Dependency Resolution):
This feature is a mod manager for the game that will automatically handles mod loading orders that avoid conflicts and apply mods in runtime. Each mod is treated as a node in a dependency graph, with interdependencies resolved through topological sorting and constraint-solving algorithms to ensure proper load order and compatibility. The system will not only handle complex mod interactions and version conflicts, but also real-time integration of new features via runtime application of mods. This will showcase sophisticated modularity and extensibility, creating a flexible ecosystem that seamlessly integrates community-driven content into the game. In addition to dependency resolution through dependency graphs, the system will also utilize advanced architectural concepts such as dynamic linking and runtime reflection.

AI:

AI for entities will be implemented using finite state machines (FSMs), allowing clear visualization and control of their behaviors. Enemy humans will have distinct roles: **Gunner**, **Shorty**, **Ranger**, **Sentry**, and **Brute**, with state transitions influenced by factors such as health and distance from the player.
Zombie AI will follow a single, simplified behavior pattern: constantly moving toward the player.
Survivor AI will operate on a basic "stay close to the player" logic to simulate following behavior.
All AI-controlled entities will utilize navigation meshes to aid in movement and pathfinding, adapted to support 2D platform-style navigation where needed.

Advanced Ballistics Simulation (Computational Physics):
This feature will leverage computational physics and numerical methods to simulate projectile trajectories, accounting for gravity, drag, and material interactions. Instead of relying on simple ray-casting for collisions in Unity's physics engine, the system will use numerical integration techniques to update a bullets position and velocity in small time increments, to simulate energy loss, penetration, and deflection upon impact. This approach requires solving differential equations and implementing specific collision response models. It aims to offer a gameplay experience that is immersive and demonstrative of advanced simulation techniques, balancing realism with computational efficiency. The approach will expand upon Unity's physics engine to incorporate these advanced features.

Equipment and Software Requirements

The only hardware requirement for the project is a computer for development that is powerful enough to run Unity. Both members already have a sufficient computer, as our program's requirements already require it.

There are several different software requirements for the project. The first one is Visual Studio (ideally 2022) for general software development, both game and infrastructure. The next is the Unity 6 game engine for game development. This will save time and ensure higher quality for the game and gives another option for the cloud solution (Unity Cloud). Unity also incorporates its own version control (Unity Version Control), that will be used as it allows effective collaboration in development. There are several other options for cloud infrastructure and usage via API besides Unity Cloud that may be used, including Steamworks SDK and the Steam Cloud, and RESTful API with a more general cloud infrastructure offered by AWS, Azure, or Google Cloud. Finally, Git and GitHub will be used for version control for all development directly outside of Unity. We already have access to all of these software requirements.

Project Advocacy

The project will be completed by a two-person team. This is feasible, as there are several areas of the project that can be independently worked on without affecting other parts of the project. Therefore, it is relatively straightforward to break down the projects into tasks and assign them to team members. For actual games, Unity Version Control can be effectively used for collaborative development.

Faculty Supervisors Consultations

- Received some feedback from Farah Hussein with the WIP of this assignment.
- Received some feedback from the first two submissions of the EOI.

- Richard Pyne mentioned that we should improve our areas of advanced computer science and make them clearer.

Feedback

- Provide more details on how you will meet the two advanced areas of computer science capstone requirement.
- EOI 1 feedback (Farah Hussein):
    - Project is suitable. Student provides initial gap analysis and identifies differentiating genre. Student plans to address this gap with the game - "game[s] that emphasize resource gathering, decision-making, and strategy, where players must carefully plan and adapt to survive."
- EOI 2 feedback (Farah Hussein):
    - Clearly articulate the potential impact of your game on the survival game genre and the gaming industry.
    - Provide specific examples of how your game will influence players and developers differently compared to existing games.
    - Prioritize the proposed features and consider aligning them with project phases when proposing the workload division plan.
    - Break down the project into manageable tasks and timelines. Include information on the resources and support you will need to complete the project successfully.
    - Since cloud technology is very important for your project, detail the technical requirements for cloud storage and how you will manage large amounts of game data. Include information on data management systems and security measures.
    - You already touched on these points. However, this part of the proposal could be more refined. A consultation with one of the cloud computing experts could be useful.
    - Prior experience and conflict of interest sections are good, but domain identification requires more details.
- EOI 2 feedback (Richard Pyne):
    - Provide more details on how you will meet the two advanced areas of computer science capstone requirement.
- EOI Part 6 submission
    - Improve AI enemies and companions to better fix advanced areas of computer science.
- EOI Part 6 (Riaan Oberholzer):
    - Given the team size and scope of the project, it's recommended to keep the graphics simple. To ensure timely completion, prioritize core features and consider simplifying or removing lower-priority elements.

Updates

- Made areas focus on mods and cloud saving
- Updated Impact sections
- Workload timeline added
- Updated areas of advanced computer science
- Updated equipment and software requirements

- Updated areas of advanced computer science again
- Tried to simply features and marked some for removal
- Remove randomness in maps and locations (Not Encounters)
- Distributed roles to Aidan as Riaan said we might have too many small details for 2 team members

Workload division Plan

Research + Implementation

Ryan	Mark	Aidan
Save-data	Modding infrastructure	Cloud infrastructure - (Mod)
Game Mechanics		

Game Feature Development

Ryan (Travel)	Mark (Combat)	Aidan (Player)
Travel: State & Cities	Weapons & Attachments	Inventory
Zones	Combat	Player Stats & Levels
State Map & City Maps	Enemies	Items
Encounters	NPCs	Looting
(looting?)		(difficulty?)

Timeline

(Weeks are estimated based off workload)

Part 1: Planning & Ideas (2 weeks)

- Team members will collaborate to discuss and design weapons, creating concept art, stats, and sound effects for each item.
- All will generate map layouts for cities and regions, ensuring randomness and diversity in environments.
- All will add to a list of all possible encounters with zones and types applied.

Part 2: Cloud and Mods setup (1 week)

- **Ryan** will lead the development of save-data systems ensuring data is saved properly.
- **Mark** will take charge of modding capabilities, ensuring that new items can be easily implemented into our game as well as having a system ready to handle it.
- **Aidan** will lead the cloud storage, finding a place to fit the saved data.

Part 3: Gameplay Start (Travel and Fighting) - Phase 1 (2 weeks)

- **Ryan** will implement the initial travel system, allowing players to select locations on the map and initiate travel between them.
- **Mark** will begin working on combat mechanics, focusing on the creation of guns and weapon attachments.
- **Aidan** will start with player stats and level up system

Part 4: Gameplay Start (Travel and Fighting) - Phase 2 (1.5 weeks)

- **Ryan** will extend the travel system to include city entry and travel.
- **Mark** will expand combat mechanics, Creating Human enemy types, adding enemy AI and enhancing overall combat dynamics.
- **Aidan** will start item creation and looting code

Part 5: Gameplay Start (Travel and Fighting) - Phase 3 (1.5 weeks)

- **Ryan** will enable encounter features and zones creation
- **Mark** will focus on designing enemy encounters, providing the necessary assets and mechanics for Ryan to add to the encounters. Also create zombie enemies.
- **Aidan** will enhance looting to work with other sections

Part 6: Actions:

- **Ryan** will focus on encounter code using Marks enemies
- **Mark** will focus on NPCs and their AI
- **Aidan** will finish inventory systems.

Part 7: Mods and Save data cont. (2 weeks)

- Both **Ryan** and **Mark** will finalize the integration of modding systems, allowing enabling and disabling.
- **Ryan** and **Aidan** will also ensure the save-data is fully integrated with cloud storage
- **Ryan**, **Mark**, and **Aidan** will continue to oversee their respective areas